

Evidence Semantics and Scanner Orchestration

A public technical paper for Security Observatory by FortMilo

Luca Pacini

The front-matter fields are reference data, so a table is easier to scan than prose.

Field	Value
Version	1.4.9
Publication date	11 August 2026
Evidence status	Claims are classified individually. No universal release-validation or environment-support claim is made.
External verification	Product-source claims remain self-attested while the supporting source is private.
Document licence	CC BY 4.0, subject to the third-party material and trademark boundary in Publication and licensing.
Published source licence	Implementation-supporting source is privately maintained. Any separately published source states its own scope and licence; Apache License 2.0 applies only where explicitly stated with that material.

Public evidence identifiers. EV identifiers are stable citable labels. A later class change is recorded against the same identifier, identifiers are never recycled, and the private assurance record maps each EV label to its exact backing artefacts.

Evidence basis notice. I separate source inspection, automated tests, controlled runtime observations, exact-candidate release validation and support. One class does not silently substitute for another. Private source and runtime identifiers are retained in the private assurance record rather than reproduced here.

Abstract

Security tooling becomes unreliable when it treats execution, source availability, observed value and interpretation as one thing. Security Observatory keeps those questions separate. A completed process does not prove that every source was available, an empty result does not prove that the relevant population does not exist, and a framework mapping does not create a compliance decision.

I built the scanner contract around one hard rule: nothing found and nothing assessed are not the same result. The product runs inside Salesforce, uses bounded asynchronous orchestration, retains sanitised evidence in the subscriber organisation and keeps source limitations visible. A user can explicitly download a sanitised CSV; that download is a separate boundary crossing and is not hidden inside an "in-org" claim.

There is one presentation detail that matters because it is easy to document badly: the implementation has internal scanner-family evidence states, while the LWC uses surface-specific operator wording. I chose that split deliberately. A control roll-up, a metric card and a source-read warning should not all be forced to show the same literal token, but none of them may say more than the underlying evidence permits.

At publication, selected false-zero and scanner-failure paths have been remediated and revalidated to the evidence classes stated in Section 3. Running-user access behaviour, complete raw-IP exclusion across every retained shape, clean-install proof and environment compatibility remain bounded separately.

1. The problem is not the count

A security dashboard can make a bad assessment look reassuring if it compresses execution, source availability and result into one number.

A displayed zero might mean the source was queried successfully and returned no matching rows. It might also mean the source could not be read. Those two situations are opposites from an evidence perspective, even though a careless implementation can make them look identical.

I therefore keep four dimensions separate. **Execution** records whether the scanner or orchestration step reached a known state. **Availability** records whether the required source was accessible and complete enough for the question. **Value** records what a successfully assessed source returned. **Interpretation** limits what the product is allowed to say about that result.

The losing design was simpler: persist a count and infer the rest later. I rejected it because once a failed acquisition has become a zero, the original uncertainty is gone. The renderer cannot recover evidence that was never retained.

2. Evidence conditions and current product presentation

The implementation does **not** expose one universal literal status vocabulary on every screen. That would be tidy documentation and worse product design. The scanner-family evidence service uses eight literal internal identifiers. I define them here once because later sections refer back to this set rather than creating a second vocabulary.

Internal identifier	Meaning in the reviewed implementation
observed	Usable family evidence was returned; at least one retained metric is observed rather than zero, or an explicit structured status resolves to observed.
zero_observed	The family was assessed and one or more returned metrics are zero, with no higher-priority limiting state.
not_returned	The family was planned for the scan but returned no metric, structured-status or coverage output from which a stronger state could be established.
unsupported	An explicit structured family status records that the relevant path or capability is unsupported.
skipped	The structured scan plan omitted the family; the family was not attempted for that run.
scanner_error	The family execution failed and only a bounded, approved failure category is retained for operator guidance.
unavailable	The assessment was attempted or coverage evidence exists, but a required source, dependency or coverage condition prevented usable complete evidence.
unknown	The service cannot establish a more specific state from reliable retained inputs; retained-evidence read failure also fails closed to unknown with a bounded reason.

I made that choice because the same evidence condition needs different context. Where a retained metric represents a successfully assessed zero, the metric value is 0. The SBS coverage surface renders the corresponding no-risk-evidence outcome as *No risk evidence surfaced*. A retained-read problem is presented as *Needs review* with a fixed warning. Those labels are allowed to differ. Their meaning is not.

This table is reference matter: it separates the evidence condition from current product presentation so the paper does not pretend that an abstract model is a literal UI dictionary.

Evidence condition	Current product presentation	Boundary / why
Observed evidence	Contextual metric or finding labels; SBS roll-up can show "Risk evidence".	Positive evidence needs the object and question around it. A generic success token would remove useful context.
Successful zero / no matching evidence	A retained metric representing a successfully assessed zero is 0; the SBS coverage surface renders "No risk evidence surfaced".	This means the intended source was successfully assessed within scope. It is not a pass and not proof that exposure cannot exist elsewhere.
Unavailable source or dependency	"Unavailable" plus bounded cause / next action; some detail copy uses phrases such as "Unavailable – source returned no value".	The assessment was attempted but the required read could not produce usable evidence. It must not become zero.
Retained-evidence read warning	The bounded read-failure path is presented as "Needs review" / a fixed warning rather than raw internal Unknown/Error.	This is not a source acquisition failure. The product cannot safely infer the retained family state, so the presentation stays conservative.
Not assessed / omitted from run	"Not assessed" or scope-specific not-run wording.	The question was not assessed for that run or scope. I keep this distinct from a failed attempted read.
Detail not retained	"Detail not retained at this evidence level", "Aggregate count only", or "No retained evidence" depending on the surface.	The scan may have observed an aggregate while deliberately not persisting lower-level detail. This is a retention choice, not source failure.
Not captured / not applicable / unsupported	Surface-specific explanatory text such as "Not captured", "Not evaluated" or "Unsupported" where appropriate.	These are different limiting reasons. I do not collapse them into one generic missing-data badge.

The rule is therefore semantic, not cosmetic: **presentation may vary by surface, but it must never strengthen the underlying evidence condition.** The product uses contextual wording because forcing every metric, finding, scanner family and SBS roll-up through one label would either hide useful context or create false equivalence.

A genuine zero has a strict precondition: the intended source was available, the query completed, the source set was complete enough for the question, and no matching records were observed. I prefer *No risk evidence surfaced within the assessed scope* to *No exposure exists*. The first sentence is accurate. The second is easier to read and much harder to justify.

3. The scanner contract and the evidence transition

3.1 Current contract

When source acquisition succeeds, the scanner may retain the observed value within its defined scope. When acquisition fails, the scanner must keep the limiting evidence condition explicit and must not substitute a zero. A composite scanner may preserve independently usable evidence, but it must label the result partial rather than manufacturing a complete total.

A family exception is isolated where that can be done safely, so independent later work continues. A Queueable transaction failure outside that isolation boundary is different: the scan is marked **Failed**, already committed evidence is retained where the failure contract allows it, and the next segment is not enqueued. The user sees a failed scan rather than a completed partial scan. Families in segments that never ran cannot receive a successful-zero result.

Known safe failure categories may be retained as bounded reason codes with fixed next actions. Raw exception text, stack traces and query text stay out of retained evidence. Unknown or unclassified conditions remain conservative and cannot resolve to a favourable zero. I would rather lose diagnostic precision than retain attacker-useful detail.

3.2 What changed after the false-zero review

Review found real acquisition-failure paths in the Connected App and Sites/Experience scanner families where a failed source could still reach zero-shaped evidence. Separate reviews found two more failure modes: scanner exceptions could lose useful bounded cause information, and one retained-evidence read failure could collapse family state into unexplained Unknown outcomes.

EV-04 accounts for that broader historical retained-read problem. The older Queueable-depth suspicion was a different question. Source inspection and one Developer Edition execution showed the intended grouped plan completing in five segments, so I retired the specific six-job hypothesis as not a defect and retained that decision as EV-09.

I fixed the confirmed product paths before I strengthened the public claims. EV-01 through EV-04 record the evidence transition. The companion delivery-method paper records the same chronology: favourable classes were frozen while the defects remained unresolved, then changed only after named remediation evidence existed.

3.3 Named evidence records

The register is reference material. Most EV records back a public claim; EV-09 documents a decision not to claim a defect after the hypothesis was disproved.

Record	Claim / decision	Evidence class	Boundary that remains
EV-01 Connected App false-zero remediation	A Connected App source failure remains unavailable; a successful genuine zero remains distinct.	Source-established and test-observed.	No runtime claim for every genuine-zero path.
EV-02 Sites/Experience unavailable-source validation	Failed Site/Experience sources retain unavailable, non-zero-shaped evidence rather than false zero.	Test-observed and runtime-observed for the controlled unavailable-source execution.	Successful-zero runtime behaviour is not claimed from that execution.
EV-03 Classified scanner-failure validation	Selected scanner exception categories retain bounded causes and safe next actions; unknown categories fail closed.	Source-established, test-observed and runtime-observed in a controlled classified-failure smoke.	Applies only to exercised or deterministically mapped categories.
EV-04 Retained-evidence read-failure validation	A retained-evidence read failure no longer collapses every family into an unexplained favourable state; readable retained evidence is preserved where the bounded layer permits.	Source-established and test-observed; healthy-path runtime-observed.	The forced degraded read path was not manufactured in live runtime, so that branch is not called runtime-observed.
EV-05 Helper-backed CSV mitigation	The reviewed JavaScript export helper neutralises enumerated spreadsheet trigger prefixes and flattens reviewed control characters.	Source-established and test-observed.	Does not cover Apex-generated or future paths unless they use the reviewed helper and tests.
EV-06 Retained comparison implementation	Comparison requires compatible retained evidence and source-organisation context in the reviewed implementation.	Source-established.	Exact-candidate and environment-specific comparison validation remain separate.
EV-07 Raw-IP exclusion boundary	Raw-IP exclusion is a design requirement; representative helper-backed export tests redact IP-looking values.	Expected.	Representative helper-backed export tests exist, but complete retained-data and export-shape validation across every path is not established.
EV-08 Running-user access boundary	User-facing evidence must respect approved sharing, CRUD and FLS behaviour.	Expected release-gate requirement.	Exact-candidate non-administrator persona validation is still required.
EV-09 Observed orchestration decision	In the observed Developer Edition execution, the reviewed grouped scanner plan completed in five Queueable segments; the historical six-job chain-depth hypothesis was rejected.	Source-established and runtime-observed for that execution.	Documents a decision not to claim a chain-depth defect; does not prove every environment or failure mode.

EV identifiers remain stable across editions. A class change is recorded against the same identifier, and an identifier is never recycled. The private assurance record maps each label to exact source, tests, deployments and runtime observations. Because that backing source is private, implementation claims remain self-attested to an external reader.

4. Evidence classes and permitted claims

The classification ladder is reference matter, so a compact table is clearer than repeating the definitions in prose.

Class	What it means	What I am allowed to claim
Source-established	The behaviour is visible in reviewed source.	Implemented in the reviewed source; runtime not implied.
Test-observed	An automated test exercised the behaviour under a controlled fixture.	Observed under that fixture.
Runtime-observed	The behaviour occurred in a reviewed Salesforce execution.	Observed in that execution and environment.
Release-validated	The exact candidate passed the defined release gates.	Validated for the stated candidate and prerequisites.
Supported	The project accepts the behaviour as part of the product contract for stated environments.	Supported only within the declared scope and prerequisites.
Expected	The behaviour is required by design, but higher evidence is incomplete.	May be stated only as an expectation or release-gate requirement.
No conclusion available	I do not hold reliable evidence for a conclusion.	No positive or negative implementation, validation or support claim.

The product-presentation token *Not assessed* is different: it denotes a scope condition in which the question was not assessed for that run or scope. It is not the evidence class *No conclusion available*.

No class is upgraded without a named artefact. For this paper, material class changes must point to an EV record, a public artefact or another named evidence record. If the backing evidence is private, I say that and keep the claim self-attested.

One runtime success is still one runtime success. It does not establish edition-wide support, clean-install behaviour or every persona. That distinction is tedious until the day it prevents a false claim.

5. Salesforce-native architecture

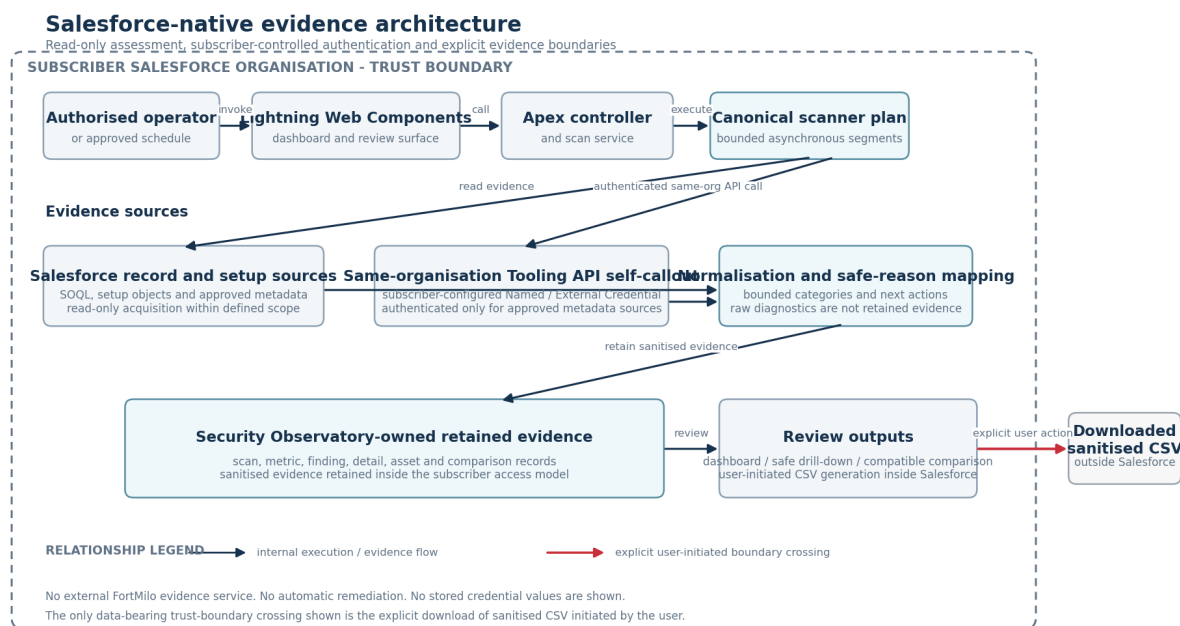


Figure 1. Salesforce-native evidence architecture. Dark arrows show internal execution or evidence flow. The red arrow is the explicit user-initiated CSV boundary crossing. The figure does not imply an external FortMilo evidence service, automatic remediation or stored credential values.

I keep the review surface, orchestration, scanners and retained product evidence in Salesforce. The Tooling API path is a same-organisation self-callout using subscriber-controlled authentication. The product does not need an external FortMilo evidence service to collect or retain scan evidence.

I also keep the scanner plan in Apex. I rejected a browser-owned scanner list because manual scans, scheduled scans and later entry points would eventually drift apart. The browser asks for a scan; Apex owns what that means.

The trust-boundary sentence needs one qualification that marketing copy often omits: user-initiated CSV download leaves Salesforce. Generation happens inside Salesforce, but once the user downloads the sanitised file, the file is outside the subscriber Salesforce trust boundary.

Running-user access is narrower than I would like to claim publicly. The design requires approved sharing, CRUD and FLS behaviour on user-facing evidence, but I will not call that environment-wide validation until the exact candidate has passed representative non-administrator persona testing.

6. Why I split the scanner plan across bounded async segments

Salesforce transaction limits make one giant assessment transaction a poor fit. The scan combines ordinary data queries, setup evidence, selected Tooling API calls and evidence persistence. I split those families across bounded asynchronous segments so a callout boundary or transaction limit in one segment does not automatically consume the whole scan.

I considered one Queueable per scanner family. It looked clean on paper and lost on transaction overhead and the risk of consuming too many chain links. I also considered one large transaction. It lost because unrelated evidence sources would share the same governor and failure boundary.

EV-09 rejects a narrower historical hypothesis: the grouped plan did not exceed the Queueable chain boundary in the observed Developer Edition run. That does not make chain depth irrelevant to a per-family fan-out design, and it does not turn the observed five-segment count into a universal product contract.

The compromise is deliberate: group compatible work, isolate callout-sensitive work, start later segments in fresh asynchronous contexts and pass forward only the orchestration context they need. Safe refactoring may change the number of segments without changing the evidence semantics.

7. Failure handling: keep the failure attached to the evidence

A scanner family may fail while later families still have independent evidence to collect. In that case I keep the family in its limiting condition and continue the independent work. The failed family does not become a successful zero just because its data structure is empty.

The Connected App false-zero defect is the concrete example. The failed acquisition used to leave an empty collection close enough to the count writer that zero-shaped evidence could escape. EV-01 now backs the narrower claim that source failure and genuine zero are distinct in the reviewed path.

Sites and Experience Cloud had a similar problem across multiple sources. EV-02 is stronger for the unavailable branch because a controlled runtime execution observed the corrected unavailable result. That runtime did not prove a successful genuine-zero branch, so the public record says so.

Scanner exceptions need a different boundary. EV-03 supports bounded categories for selected known failures and a generic fail-closed category for everything else. I rejected retaining raw exception messages because they are operationally convenient and security-poor. The admin gets a safe cause and next action; the raw diagnostic stays in the engineering channel.

Retained-evidence read failures are narrower again. EV-04 proves the layered read behaviour in source and tests, and the healthy read path has runtime observation. I did not manufacture a live degraded read failure merely to make the evidence table look stronger. The forced degraded branch remains test-observed.

8. Source availability drives the condition; the surface decides the words

The evidence model and the UI have different jobs. The evidence layer must preserve the reason a result is limited. The UI must explain that reason in the context the operator is looking at. I deliberately did not make those two layers share one literal label set.

Section 2 defines the literal scanner-family evidence identifiers. This section does not create a second enumeration. The SBS coverage surface translates those evidence conditions into operator wording such as *No risk evidence surfaced* and *Needs review*. Metric and drill-down surfaces use other bounded phrases, including *Aggregate count only*, *Detail not retained at this evidence level* and *Unavailable – source returned no value*.

I chose contextual presentation because a literal "Zero Observed" badge on every surface would be technically neat and operationally poor. A security reviewer wants to know whether there is evidence to review, whether the source failed, or whether detail was intentionally not retained. The implementation can vary the words as long as it does not vary the evidence boundary.

Unavailable therefore means an attempted read could not obtain usable evidence. Not assessed means the question was not assessed for that run or scope. Not retained means lower-level detail was deliberately not persisted. Those are different reasons, even if all three can produce an empty-looking panel.

9. Sanitisation and export boundaries

Security Observatory retains review evidence, not a forensic dump. Tokens, session identifiers, raw secrets, private keys and certificate bodies are outside the retained review contract. Raw exception text and raw queries are also excluded from customer-facing evidence.

Raw IP handling is a narrower claim. I want the product boundary to exclude raw IP values, but I do not have complete retained-data-shape proof across every current and future path. EV-07 therefore stays at Expected for the universal exclusion claim. Representative helper-backed export tests do redact IP-looking values; that does not justify rewriting the broader class.

9.1 Spreadsheet formula-trigger mitigation

EV-05 covers the reviewed JavaScript export helper. The helper quotes cells, doubles embedded quotes, flattens tabs and line breaks, and prefixes an apostrophe when the first meaningful character is =, +, - or @.

That is useful mitigation. It is not a universal CSV-safety claim. I rejected the phrase *formula-injection-safe by default* because it says more than the evidence does. Apex-generated exports and future paths need their own review unless they are routed through the same helper and regression tests.

10. Subscriber-controlled Tooling API self-callout

Some metadata evidence requires the subscriber organisation to call its own Salesforce API. The approved pattern uses Named Credential and External Credential components with a subscriber-controlled principal. The package does not carry an authenticated principal on the customer's behalf.

If that prerequisite is missing or the principal lacks the required access, the evidence condition is **Unavailable** and current surfaces present a bounded cause and safe next action. I rejected Not assessed for this condition because the scanner did

attempt an assessment and the limiting dimension is source availability, not selection.

I also reject credential probing as a verification shortcut. The Observatory is read-only and advisory. It inventories and evaluates bounded configuration evidence; it does not try to prove a credential works by using it against unrelated resources.

11. Historical evidence and comparison

A historical scan should mean what it meant when it ran. Current live state must not be used to reinterpret an older retained result.

Comparison therefore operates on retained evidence from compatible scans. The implementation represented by EV-06 checks source-organisation context and retained evidence depth before it compares results. If the evidence depth is incompatible, comparison is unavailable rather than pretending that omitted detail was resolved.

I keep this claim at Source-established. The comparison implementation exists, but this paper does not present an exact-candidate, cross-environment release-validation claim for the whole comparison surface.

12. Framework mapping supports review, not compliance

Security Observatory can attach Security Benchmark for Salesforce (SBS) references to retained evidence. I use that layer to help a reviewer organise evidence, not to manufacture a compliance result.

A control reference does not prove that the control is satisfied. A mapped scanner family can be unavailable, partial or outside automated coverage. The retained mapping therefore carries the registry version and local mapping revision used for the scan so later catalogue changes do not rewrite historical evidence.

Evidence class: Source-established. Reviewed local mapping metadata identifies the catalogue as mapping set M1, revision 1; a reviewed retained scan record reports SBS registry value v0.4.0. These are local version-bound mapping inputs, not certification statements. This paper does not establish the upstream asset provenance or equivalence to the current upstream release tag, and it does not publish a control-count claim.

13. Public wording discipline

These are the wording choices I use when a shorter phrase would outrun the evidence.

Avoid: "The org is secure." Prefer: "The assessed source returned the stated evidence within the documented scope."

Avoid: "No exposure exists." Prefer: "No risk evidence surfaced within the assessed scope" or the surface-specific zero count.

Avoid: Generic Unknown/Error for an unavailable source. Prefer: Unavailable with a safe cause where known. Reserve the retained-read warning presentation for EV-04.

Avoid: All missing evidence is Not assessed. Prefer: distinguish unavailable source, not assessed scope, not retained detail and not captured fields by the actual limiting reason.

Avoid: "Formula-injection-safe CSV export." Prefer: the reviewed helper-backed paths mitigate the enumerated formula-trigger prefixes and control characters.

Avoid: "Raw IP values are universally excluded." Prefer: raw-IP exclusion is a design requirement; complete retained-data-shape validation is still bounded by EV-07.

Avoid: "FLS-gated controls are fully validated." Prefer: running-user sharing, CRUD and FLS remain subject to EV-08 exact-candidate persona validation.

Avoid: "Evidence never leaves Salesforce." Prefer: there is no automatic external FortMilo evidence egress; a user-initiated sanitised CSV download is an explicit boundary crossing.

14. Current limitations

I have fixed the defects behind EV-01 through EV-04. That does not make the whole product release-validated. The current limits that still matter are these:

- Exact-candidate non-administrator persona validation is still required before I make an environment-wide sharing, CRUD or FLS claim.
- Complete retained-data and export-shape validation for the universal raw-IP exclusion boundary is not established.
- The forced degraded retained-evidence read branch is test-observed rather than runtime-observed.

- Like-for-like comparison is source-established here; exact-candidate and representative-environment validation remain separate.
- Clean-install and package-install proof for the exact public candidate remain separate and are not claimed here.
- A successful execution in one Salesforce organisation does not establish edition-wide compatibility.
- A Queueable transaction failure outside per-scanner isolation marks the scan Failed, retains already committed evidence where the failure contract allows it and does not enqueue later segments. Families that never ran cannot receive a successful-zero result.
- The product remains advisory and read-only; it does not remediate findings or prove that a retained OAuth authorisation is a currently usable token.
- Some Tooling-backed evidence depends on subscriber-controlled authentication and permissions.
- Product-source implementation claims remain self-attested while the supporting source is private.

These are compromises, not decorative caveats. I would like the evidence classes to be higher. They are not higher until the corresponding gate has actually been run.

15. Conclusion

A security dashboard is trustworthy only when it can explain the path from source to statement. The useful property is not the number of counts on screen. It is the refusal to turn missing evidence into favourable evidence.

That rule shaped the product and it also exposed defects in the product. The Connected App and Sites/Experience failures were not theoretical; they showed that an empty data structure can become a false zero if the scanner contract is not carried all the way to persistence. I fixed those paths, recorded the evidence transition and kept the remaining claims at their actual classes.

The same discipline applies to presentation. I do not require every screen to use the same words, because a metric, a finding and an SBS roll-up serve different operator questions. I require every screen to preserve the same evidence boundary. That is why the implementation can say *No risk evidence surfaced* in one place and show a numeric zero in another without turning either into a pass.

The paper is more useful when it describes the system that actually exists rather than a cleaner vocabulary I wish existed.

Closing principle: nothing found and nothing assessed are not the same result.

References

1. Salesforce Developers, Secure Apex Classes. <https://developer.salesforce.com/docs/platform/lwc/guide/apex-security.html>
2. Salesforce Developers, Call APIs from Apex. <https://developer.salesforce.com/docs/platform/lwc/guide/data-api-calls-apex.html>
3. Salesforce Developers, Get Started with Named Credentials.
<https://developer.salesforce.com/docs/platform/named-credentials/guide/get-started.html>
4. Salesforce Developers, Named Credentials Glossary.
<https://developer.salesforce.com/docs/platform/named-credentials/references/named-credentials-reference/nc-glossary.html>
5. Salesforce Developers, Package Named Credentials.
<https://developer.salesforce.com/docs/platform/named-credentials/guide/nc-package-credentials.html>
6. Salesforce Developers, Populate External Credential Principals.
<https://developer.salesforce.com/docs/platform/named-credentials/guide/nc-populate-external-credentials.html>
7. Security Benchmark for Salesforce. <https://www.securitybenchmark.org/>
8. Security Benchmark for Salesforce documentation. <https://docs.securitybenchmark.org/>
9. Creative Commons, Attribution-ShareAlike 4.0 International. <https://creativecommons.org/licenses/by-sa/4.0/>
10. Creative Commons, Attribution 4.0 International. <https://creativecommons.org/licenses/by/4.0/>
11. Apache Software Foundation, Apache License, Version 2.0. <https://www.apache.org/licenses/LICENSE-2.0>
12. FortMilo, Orchestrating AI for Secure Software Delivery, 2026.
<https://fortmilo.co.uk/documents/orchestrating-ai-for-secure-software-delivery.pdf>

Each URL is printed in full so the references remain usable in printed copies.

Publication and licensing

I am the author of this paper and trade as FortMilo. Public location: United Kingdom.

Source supporting the implementation claims in this paper is privately maintained; those claims remain self-attested unless a specific public artefact is cited. The installable package and any source code published separately are distinct publication artefacts and are not equivalent in extent. Any separately published source states its own scope and licence. Publication of source code does not by itself represent package availability, release validation or installation readiness. I retain the exact private baseline and backing artefacts for the public evidence records in the private assurance record.

Copyright 2026 Luca Pacini. This paper is licensed under Creative Commons Attribution 4.0 International (CC BY 4.0). The licence applies to FortMilo-authored text and figures in this paper. It does not relicence third-party trademarks or cited material.

Security Benchmark for Salesforce material remains subject to its own attribution and licence terms. No SBS control text, audit procedure or remediation content is reproduced or adapted in this paper.

FortMilo is a Salesforce Partner. Partner status is an organisational relationship only; it does not imply a particular partner-programme tier or entitlement, nor does it imply AppExchange listing, Salesforce security review or approval for customer distribution of Security Observatory. Security Observatory is independently developed and is not endorsed by Salesforce, Inc. Salesforce is a trademark of Salesforce, Inc.